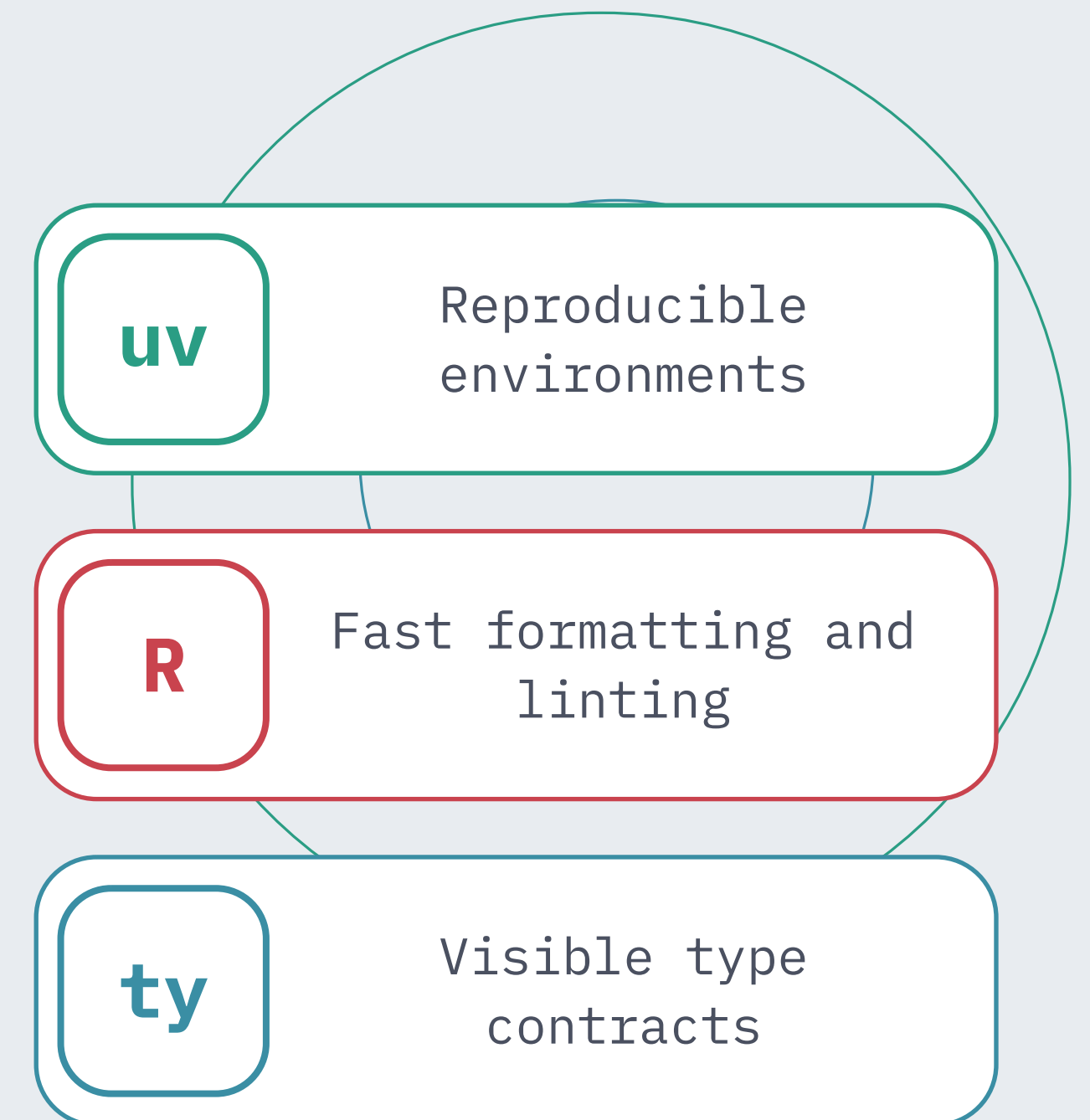


Why uv, ruff, and ty are mandatory

A pragmatic Python tooling baseline for fast, safe projects

For teams that want reproducible setup, fast feedback, and safer changes without turning reviews into tooling debates.



The baseline promise

uv

One tool for Python versions, virtualenvs, dependencies, lockfiles, and scripts.

ruff

Formatter and linter fast enough to run constantly, not just in CI.

ty

Static type checks that catch integration bugs before runtime.

Together: reproducible setup, instant style feedback, and safer refactors.

Setup

One command gets a new checkout ready.

Review

Formatting and linting stop being subjective.

Change

Types expose contracts before production.

Why mandatory, not optional?

Optional tools create optional quality

If teams can skip the baseline, every project develops a different failure mode.

First five minutes should be identical

Install, sync, lint, typecheck, and test should not require local archaeology.

Inconsistency compounds

Onboarding, CI, reviews, and releases all pay for tool drift.

Automation beats taste debates

Make quality enforceable by command instead of preference.

Mandates turn team preference into repeatable project behavior.

uv: make setup boring

- Creates and manages environments quickly.
- Locks dependencies for repeatable installs.
- Runs commands inside the project environment.
- Reduces pip, venv, pyenv, and poetry drift.

project shell

```
uv python install 3.12
uv init billing-api
uv add fastapi pydantic
uv add --dev pytest ruff ty
uv run pytest
```

The boring path is the scalable path: install, sync, run, repeat.

uv gives every contributor the same path

project shell

```
# pyproject.toml
[project]
requires-python = ">=3.12"
dependencies = ["fastapi", "pydantic"]

[dependency-groups]
dev = ["pytest", "ruff", "ty"]
```

Why it matters

A lockfile plus `uv sync` turns onboarding and CI from archaeology into a deterministic step.

ruff: one fast gate for style and defects

project shell ● ● ●

```
uv run ruff format .  
uv run ruff check . --fix
```

- Replaces slow stacks of formatters and linters.
- Finds unused imports, unsafe patterns, and style drift.
- Auto-fixes the boring stuff before review.
- Makes formatting a command, not a conversation.

Reviews should focus on design and behavior, not import order.

ruff catches real bugs early

project shell ● ● ●

```
from pathlib import Path

def load_config(path: str) -> dict[str, str]:
    ...file_path = Path(path)
    ...if not file_path.exists:
    .....return {}
    ...return parse_config(file_path.read_text())
```

Review saved

Ruff flags suspicious callable misuse, undefined names, unused imports, and common review distractions.

ty: make contracts visible

- Checks function boundaries and data flow.
- Protects refactors across files.
- Documents intent without prose comments.
- Complements tests; it does not replace them.

project shell

```
def total_cents(items: list[int]) -> int:  
    ...return sum(items)  
  
subtotal = total_cents([1299, 2500])  
label: str = subtotal # type error
```

Types make incorrect wiring visible before runtime paths are exercised.

A practical mandatory workflow

project shell

```
uv sync
uv run ruff format .
uv run ruff check .
uv run ty check
uv run pytest
```

Mandate

Run this locally,
optionally in pre-commit,
and always in CI before
merge.

1

local path

1

pre-commit

1

CI gate

**Same command path locally, in pre-commit,
and in CI.**

Common objections

Too strict?

Start small, then tighten. Mandatory does not mean maximal on day one.

Too new?

Pin versions and keep upgrade PRs explicit.

Types slow us down?

Unclear contracts slow you down later; annotate boundaries first.

CI already tests?

Tests check examples; lint and types check broad mistake classes.

Adopt progressively, but make the baseline non-negotiable.

Adoption checklist

01

Declare

Add ``requires-python``, dependencies, and dev dependency group to ``pyproject.toml``.

02

Lock

Commit the lockfile and document ``uv sync`` as the setup path.

03

Gate

Add ``ruff format``, ``ruff check``, ``ty check``, and tests to CI.

04

Prevent

Fix existing findings once; prevent regressions forever.

One baseline, one CI gate, one documented path for every project.

Takeaway

uv
environment

ruff
quality

ty
contracts

Make the reliable path the default path.

Mandatory tooling is not bureaucracy. It is the cheapest way to make every Python project easier to trust, review, onboard, and change.

uv + ruff + ty = reproducible setup, fast feedback, safer change.